

SynFinder: transparent discovery of synthetic data generation methods for biomedical research

First Author¹ and Last Author^{ib} ²

BioHackathon series:

[DBCLS BioHackathon 2026](#)

Matsuyama, Japan, 2026

[synthetic data generator finder](#)

Submitted: 18 Sep 2026

License:

Authors retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC-BY](#)).

Published by [BioHackrXiv.org](#)

1 First Affiliation 2 ELIXIR Europe 

Abstract

Selecting a synthetic data generator requires matching research objectives to data structure, privacy requirements, and practical constraints. During DBCLS BioHackathon 2026, we developed SynFinder, a catalog-driven tool for discovering generation methods and existing synthetic datasets. SynFinder records method capabilities, computational requirements, software maturity, privacy annotations, limitations, and evaluation guidance in a structured schema. It uses hard constraints to identify eligible methods and a purpose-aware weighted score to produce explained shortlists. Recommendations include caveats, illustrative output formats, source links, and a comparison of scoring axes. The prototype contains 66 rankable methods or method families across ten data types, three separately classified frameworks, and six ready-made datasets. A catalog audit found caveats, evaluation guidance, and output previews for every rankable entry, but numerical scale bounds for only three. All 102 selected automated checks passed. The software is accessible through a browser application, a Python web service, and a command-line interface. SynFinder provides an inspectable starting point for method selection; its heuristic scores and curated annotations require independent evaluation before they can be interpreted as evidence of recommendation quality.

Introduction

Synthetic data generation encompasses approaches with different assumptions and intended uses. For example, synthpop provides routines for synthesizing microdata in R (Nowok et al., 2016), conditional generative adversarial networks address mixed-type tabular data (Xu et al., 2019), and Synthea simulates patient histories and electronic health records (Walonoski et al., 2018). These examples illustrate why the label “synthetic data generator” alone provides insufficient information for selecting a tool.

A researcher may need records for testing a pipeline, repeated observations for developing a prediction model, or simulated genomic data for benchmarking. Selection also depends on supported variable types, computational resources, required expertise, and the properties the researcher wants the generated data to preserve. A useful discovery resource should therefore describe both a method’s intended applications and the conditions under which it may be unsuitable.

Libraries such as synthcity provide generation and experimentation facilities across several data modalities (Qian et al., 2023). SynFinder addresses the preceding discovery step: expressing a research task, identifying compatible candidates, and understanding why each candidate appears. Its contribution is a shared descriptive schema linked to an explicit selection procedure and method-specific caveats. We do not claim a new generation algorithm or superior synthetic-data performance.

We built SynFinder during the DBCLS BioHackathon 2026 week in Matsuyama, Japan (“SynFinder: source code and method catalog”, 2026). This report presents three outputs: a

structured catalog of generation approaches and datasets; a reproducible filtering and ranking workflow; and interfaces that expose recommendations, limitations, and supporting resources. We assess catalog coverage and software behavior, and identify the evidence still needed to evaluate selection quality.

Methods

Development scope and catalog construction

The hackathon implementation separates catalog records, researcher requirements, selection logic, and presentation. Methods are stored as YAML files, with Pydantic models defining the record structure and a shared taxonomy defining controlled terms. A record can describe an individual implementation or a broader method family; framework records are explicitly identified and excluded from generation-method ranking. Consequently, the number of entries is not a count of distinct algorithms.

The catalog records paper, code, and documentation links alongside structured annotations. Contribution instructions require concrete caveats, evidence for numerical scale bounds, and disclosure when a contributor authored a proposed method. The repository supports entry creation through an interactive command and submission through pull requests or issue forms. These mechanisms define the contribution workflow; they do not demonstrate that every existing annotation underwent independent review. The hackathon catalog should be treated as a prototype collection rather than the outcome of a systematic literature search.

Features recorded for generation methods

The method schema contains 26 top-level fields. Tables below describe all fields, including their nested attributes, and distinguish features used for filtering or scoring from descriptive information. This distinction matters because collecting a feature does not imply that the ranking engine uses it.

Table 1: Identity, applicability, and practical features recorded for generation methods.

Feature and schema fields	Information represented	Use in SynFinder
Identity: <code>id</code> , <code>name</code> , <code>aliases</code>	Stable identifier, display name, and alternative names	Record identification and cross-references; aliases do not affect ranking
Approach: <code>family</code> , <code>is_framework</code>	Method family and whether the entry represents a framework	Family is descriptive; frameworks are excluded from ranked methods
Domain: <code>domains</code>	Biomedical, social sciences and humanities, or general applicability	Hard filter; a general-purpose annotation matches any selected domain
Data structure: <code>data_types</code>	Cross-sectional or longitudinal tables, coded event sequences, time series, survival, text, images, graphs, genomic data, or single-cell omics	Hard filter and catalog coverage reporting
Variable support: <code>variable_types</code>	Continuous, categorical, ordinal, count, datetime, and free-text variables	Hard filter when requested variable types are supplied

Feature and schema fields	Information represented	Use in SynFinder
Privacy: <code>formal_dp</code> , <code>dp_mechanism</code> , <code>requires_no_source_data</code>	Formal differential privacy annotation, mechanism description, and whether generation requires no source records	Privacy eligibility uses the two Boolean annotations; mechanism text documents the claim
Computing: <code>compute</code>	CPU-compatible, GPU-recommended, or GPU-required	A CPU-only requirement excludes GPU-required entries
Software access: <code>license</code> , <code>language</code>	License identifier and implementation language	Open-license filtering uses a configured allowlist; language is descriptive
Currency: <code>last_updated</code>	Recorded update year	Displayed as practical metadata; not used to calculate the score

The taxonomy groups approaches into generative adversarial networks, variational autoencoders, diffusion models, probabilistic graphical models, marginal-based methods, copulas, sequential tree methods, language models, agent simulation, rule-based generation, resampling, and toolkits. These categories organize the collection without implying equivalent assumptions or performance within a family.

Table 2: Scientific fit, evidence, and explanatory features recorded for generation methods.

Feature and schema fields	Information represented	Use in SynFinder
Application: <code>purposes</code>	Open release, pipeline testing, machine-learning augmentation, statistical replication, causal inference, education, benchmarking, and imbalance correction	Purpose selection and scoring
Intended fidelity: <code>preserves</code>	Marginals, joint correlations, temporal dynamics, causal structure, and rare events	Fraction of requested properties annotated as supported
Scale: <code>scale.min_rows</code> , <code>scale.max_rows</code> , <code>scale.max_cols</code> , <code>scale.evidence</code>	Recorded row and column bounds with supporting evidence text	Row bounds affect scale scoring; column bounds and evidence are descriptive
User requirements: <code>expertise</code>	Low, medium, or high required expertise	Scored against the researcher’s stated expertise
Maturity: <code>maturity.maintained</code> , <code>maturity.last_release</code> , <code>maturity.implementation_quality</code>	Maintenance status, release information, and reference-only, research-code, or production-ready classification	Maintenance and implementation quality affect scoring; release text does not
Governance: <code>governance.acceptance_evidence</code> , <code>governance.known_deployments</code>	Recorded acceptance evidence and examples of use	Presence affects the optional governance score; record counts also break ties
Limitations: <code>caveats</code>	Method-specific failure modes and applicability qualifications	Included in explanations; at least one caveat is required

Feature and schema fields	Information represented	Use in SynFinder
Evaluation: <code>evaluation</code>	Suggested checks for generated data	Included as guidance; SynFinder does not execute these checks
Sources: <code>links.paper</code> , <code>links.code</code> , <code>links.docs</code>	Publication, implementation, and documentation links	Supporting resources presented with recommendations
Dataset relationships: <code>related_datasets</code>	Identifiers of associated dataset records	Catalog cross-references; not the basis of intake-based dataset matching
Output illustration: <code>output_preview.format</code> , <code>output_preview.note</code> , <code>output_preview.preview</code>	Preview type, explanatory note, and illustrative content	Presentation of expected output structure, not generator execution

Preservation tags summarize catalog judgments about intended capability; they are not measured guarantees for a new dataset. Similarly, governance records describe evidence recorded by contributors rather than certification. Suggested evaluation procedures vary by entry. For example, the CTGAN record lists distributional comparisons and train-on-synthetic/test-on-real assessment, whereas the Synthea record emphasizes compatibility with expected code systems and message formats. These examples illustrate the different guidance stored in the catalog, not evaluation conducted by SynFinder.

The method loader checks controlled terms and rejects duplicate method identifiers. The schema requires a named mechanism when `formal_dp` is true and an evidence field when any numerical scale bound is present. Catalog tests additionally check source-link presence, resolution of related-dataset identifiers, and the presence and type of output previews. These are structural and consistency checks. They cannot establish the correctness of an annotation or whether a source supports its interpretation.

Researcher intake and hard constraints

Four inputs are required: research domain, data type, intended purpose, and whether privacy measures are required. Optional inputs specify compute availability, researcher expertise, expected row count, variable types, desired preservation properties, an open-license requirement, and a request for governance evidence. The intake describes requirements; it does not upload source records or train generators.

After framework entries are removed, hard filters assess domain, data type, privacy annotations, GPU requirements, variable support, and license compatibility. A candidate fails variable compatibility when it lacks any requested variable type. Under a CPU-only requirement, GPU-recommended entries remain eligible while GPU-required entries are excluded. License filtering checks a fixed allowlist rather than conducting a legal analysis. The system reports the first failed constraint for each excluded method.

For the privacy-required setting, a candidate remains eligible if it is tagged as formally differentially private or as requiring no source records. These alternatives represent distinct conditions. The no-source-records annotation is not a differential privacy guarantee, and the formal-DP flag does not verify a particular training run or privacy budget. The privacy-not-required setting imposes no privacy filter and therefore retains both privacy-annotated and other methods.

Purpose-aware ranking and explanations

Each eligible method receives a weighted mean of its active axis scores:

$$S(m, q) = \frac{\sum_{a \in A(q)} w_a s_a(m, q)}{\sum_{a \in A(q)} w_a},$$

where m is a method, q is the intake, $A(q)$ denotes active axes, w_a is a configured weight, and s_a is a compatibility score. Purpose and maturity are always active. Preservation, expertise, and scale activate only when their inputs are supplied; governance activates when requested. Unanswered optional axes are omitted from the denominator.

Table 3: Axis definitions and default weights at the evaluated revision.

Axis	Weight	Scoring rule
Purpose	3.0	1 for a listed purpose; otherwise 0
Preservation	2.5	Number of requested properties annotated as supported divided by number requested
Expertise	1.5	1 when required expertise does not exceed the user's level; 0.5 for one level above; 0 for two levels above
Maturity	1.5	0.3 for reference-only, 0.6 for research code, 1 for production-ready; multiplied by 0.5 when unmaintained
Scale	1.0	1 within recorded row bounds or without row bounds; 0.4 below a minimum or above a maximum by at most twofold; 0.1 above a maximum by more than twofold
Governance	1.0	1 when any acceptance evidence or deployment is recorded; otherwise 0.2

If any eligible method matches the requested purpose, only purpose-matching candidates are retained for sorting. Otherwise, eligible candidates are returned with an explicit no-purpose-match warning. Thus, purpose is both an axis and a candidate-selection rule; it is not traded freely against the remaining features. Within a purpose-matching candidate set, its score is constant.

Candidates are sorted by total score, implementation-quality score, and the combined count of governance evidence and deployment records. These tie-breakers apply even when the user has not requested governance evidence. Remaining ties preserve catalog loading order. The default shortlist contains five entries; the web workflow additionally exposes up to 19 further ranked candidates. Scores are heuristic compatibility summaries, not probabilities of suitability, privacy, or data quality.

Explanations present up to two favorable axes with scores of at least 0.75, ordered by weighted contribution. When any axis scores below 0.6, one weakness is selected by its weighted

contribution. Catalog caveats and evaluation suggestions accompany this explanation. A comparison table exposes per-axis scores, and the report lists hard-filter exclusions. Empty-result messages distinguish an uncovered data type from a covered type with no eligible method. Purpose mismatches and candidates beyond the displayed shortlist are not reported as hard-filter exclusions.

Ready-made dataset discovery and output previews

Dataset records contain identity (`id`, `name`), applicability (`domains`, `data_types`, `purposes`), size (`n_records`), privacy annotations (`formal_dp`, `dp_mechanism`), provenance (`generated_by`), access conditions (`access_conditions`), license (`license`), realism limitations (`realism_caveats`), and source links (`links.paper`, `links.code`, `links.docs`). Dataset size is a descriptive string, allowing units such as patients or records rather than forcing a common row count. At least one realism caveat is required.

Intake-based dataset discovery matches domain, data type, and purpose; an empty purpose list does not exclude a record. Matching datasets appear before the method shortlist. A separate browser view filters datasets by domain and data type, while the command-line interface and server API also support license filtering. The dataset privacy fields are stored but do not participate in matching. Consequently, a suggested dataset remains a resource to inspect, rather than a resource certified to satisfy every method-selection constraint.

Method previews cover tabular CSV examples, event sequences, genomic matrices, image specifications, text, and graph edge lists. Every rankable entry has a preview in the evaluated catalog. Previews illustrate output structure and carry an explicit disclaimer that they were not generated by the method. Imaging previews describe dimensions and format rather than presenting fabricated medical images.

Interfaces, deployment, and reproducibility

SynFinder uses one Python ranking implementation across its command-line interface, FastAPI service, and static browser deployment. The browser deployment uses Pyodide to load the package and catalog and execute selection locally after the initial resources have downloaded. The repository includes a static-site builder, GitHub Pages deployment, container support, and a Codespaces configuration. This architecture shares the selection logic across access routes without requiring a separately implemented JavaScript ranking engine.

The web application provides a structured intake form, dataset browsing, recommendation cards, additional candidates, comparison scores, and downloadable Markdown and HTML reports. Reports retain the intake, shortlist, caveats, evaluation guidance, and exclusions. The command-line interface supports recommendation generation, dataset browsing, and catalog-entry creation. Metadata such as implementation language, update year, and maintenance status helps users inspect practical requirements.

Core recommendations require no external model service. An optional language-model narration module is connected to the retained Streamlit prototype, rather than the current FastAPI or static-browser recommendation paths. Its prompt requests preservation of caveats and restriction to supplied facts, with deterministic text used when narration is unavailable. Prompt instructions alone do not guarantee faithful rewriting.

Evaluation procedure

We evaluated revision 89e0923fb1d5dcac5e676d28f207602f011ee47d. We loaded the catalog to count entries, modalities, families, and selected metadata fields, then executed three fixed intakes with `top_n=3` and the repository's default weights. Inputs, outputs, scores, and catalog counts are recorded in `paper/catalog-audit.json` in the report repository.

We also ran 15 selected test modules covering catalog integrity, predefined scenarios, hard filters, scoring, dataset matching, method and dataset schemas, catalog loading, intake handling, explanations, previews, reports, the command-line interface, contribution helpers, and optional-narration logic. The exact command is retained in the accompanying review notes. Tests of narration logic do not constitute evaluation of live model responses. This evaluation assesses software consistency and catalog structure; it does not measure the utility or privacy of generated data.

Results

Catalog breadth and completeness

The catalog contains 69 method and framework records: 66 rankable entries across 11 method families, and three frameworks (medigan, SDV, and synthcity). Six ready-made dataset records are included separately. The taxonomy covers ten data types, but entry density varies substantially.

Table 4: Rankable catalog entries by supported data type. Entries may support more than one type, so counts are not additive.

Data type	Entries
Cross-sectional tabular	30
Longitudinal tabular	12
Time series	9
Coded event sequences	8
Genomic	5
Single-cell omics	5
Text	5
Images	4
Graphs	4
Survival	1

All 66 rankable entries contain caveats, evaluation guidance, and output previews. Publication links are recorded for 62 entries, code links for 61, and documentation links for 12. Implementation language is populated for 62 entries and update year for 58. Thirty-one entries contain at least one governance or deployment record, while only three have numerical scale bounds. Nine entries carry a formal-DP annotation and seven carry a no-source-records annotation. These are counts of recorded information, not independent confirmation of its accuracy.

The audit therefore supports two conclusions about the prototype: it provides explanatory content throughout the rankable catalog, and the evidence available for some scoring dimensions is sparse. In particular, scale metadata cannot currently support a well-evidenced comparison for most entries.

Software checks and illustrative shortlists

All 102 selected automated tests passed. Predefined cases exercise privacy constraints, resource constraints, purpose selection, and several modalities. Other tests assess record validity, omitted-input handling, explanations, previews, reports, contribution helpers, and command-line behavior. The development scenarios help guide the encoded rules, so passing them demonstrates regression consistency rather than independent recommendation accuracy. The complete suite, deployed browser runtime, live links, container deployment, and live external-model narration were not evaluated in this run.

Table 5: Executed selection examples. All use the biomedical domain and privacy-not-required setting; omitted optional inputs remain unspecified.

Task and requirements	Top three candidates, in returned order	Hard-filter exclusions
Pipeline testing; cross-sectional tabular; CPU only; low expertise	synthpop; metasyn; Rule-based expert simulation	43
Machine-learning augmentation; coded event sequences	CEHR-GPT; EHRDiff; EVA	58
Benchmarking; genomic data	msprime; SLiM; HAPNEST	61

Exclusions are counted out of 66 rankable entries. Subsequent purpose selection and shortlist truncation can remove additional eligible candidates from view. In the first scenario, all three displayed candidates receive a score of 1.0. This is full compatibility on the active encoded axes, not evidence of equivalent or perfect performance. The examples demonstrate reproducible selection behavior across tasks; they do not establish that the returned order is preferred by researchers.

Discussion

Contribution and interpretation

The hackathon produced a functioning discovery workflow linking a structured feature catalog to an explicit selection procedure. Its practical contribution is to make method selection inspectable: a researcher can connect requirements to exclusions, compare scored dimensions, review limitations, and follow source links. Dataset discovery offers an additional route when an existing resource may meet a task. The catalog can grow independently of the ranking implementation, although new scientific concepts may still require taxonomy and logic changes.

The schema also reveals what the prototype does not yet know. Descriptive features such as language, update year, release text, and maximum column count do not directly affect the score. Recorded preservation capabilities, maturity classifications, and governance narratives remain judgments rather than standardized measurements. A shortlist should therefore guide further inspection and task-specific evaluation.

Limitations of the catalog and ranking

Coverage is uneven and no systematic search protocol, screening record, or inter-reviewer agreement assessment is established by the repository. Ten supported modalities should not be interpreted as comparable depth across them. Versioned field-level citations, annotation dates, reviewer identities, and a way to distinguish unknown from unsupported capabilities would improve traceability.

Scale illustrates a more specific evidence problem. The current score assigns full compatibility when row bounds are absent, which applies to 63 of 66 rankable entries. Moreover, the CTGAN record derives bounds from benchmark dataset sizes (Xu et al., 2019); an evaluated dataset size does not by itself establish a minimum training requirement or maximum supported size. Future curation should separate observed benchmark configurations from demonstrated operating limits, and the ranking should represent missing evidence explicitly.

The weights and score mappings are heuristic. Binary purpose and preservation tags simplify conditional suitability, and purpose matching constrains the candidate set before sorting. With

only the four core answers, purpose and maturity are the only active scoring axes; among purpose matches, ordering is therefore driven by maturity and tie-breakers. Governance evidence counts can favor better-documented entries without indicating greater scientific suitability. These design choices warrant sensitivity analysis and comparison with independent judgments.

Privacy annotations also have limited scope. A formal-DP flag lacks structured privacy budgets and run-specific verification, while a no-source-records tag describes a different basis for eligibility. Dataset matching does not enforce either condition. Accordingly, the tool cannot certify a release decision or the privacy properties of data produced with a recommended implementation. Similarly, suggested quality checks and illustrative previews should not be mistaken for measured results.

Evaluation and next steps

The reported tests establish consistency with selected software requirements; they do not demonstrate reduced search time, improved choices, or successful downstream studies. A next evaluation should use tasks and expert reference judgments developed independently of the regression scenarios. It could assess shortlist suitability, inappropriate inclusions and exclusions, understanding of caveats, and time required to identify a candidate. Weight sensitivity and ablation of purpose, maturity, and governance rules would clarify what drives ranking differences.

Catalog review is equally important. Priority work includes reviewing evidence behind suitability tags, broadening sparsely covered modalities, distinguishing unknown values from favorable evidence, and documenting the provenance of annotations. Testing the deployed browser workflow and checking agreement across interfaces would complement the software checks reported here. These steps would move the hackathon prototype toward an evaluated discovery resource while preserving its inspectable decision procedure.

Conclusion

SynFinder combines structured descriptions of synthetic data methods with constraint filtering, purpose-aware ranking, dataset discovery, and explicit explanations. Developed during DBCLS BioHackathon 2026, the prototype includes 66 rankable entries across ten data types and passes 102 selected automated checks. Its contribution is a working, inspectable selection workflow. Catalog evidence review and independent evaluation remain necessary to establish the quality and usefulness of its recommendations.

Availability

Application: <https://sunchang0124.github.io/SynFinder/>.

Source code and catalog, distributed under the MIT license: <https://github.com/sunchang0124/SynFinder>.

Manuscript and catalog audit: <https://github.com/biohackathon-japan/BH26-synthetic-data-generator-fi>

Acknowledgements

[TODO: confirm contributor names, affiliations, funding, and acknowledgement text.]

Author contributions

First Author: Writing – original draft; Last Author: Conceptualization, Writing – review & editing.

References

- Nowok, B., Raab, G. M., & Dibben, C. (2016). synthpop: Bespoke creation of synthetic data in R. *Journal of Statistical Software*, 74(11), 1–26. <https://doi.org/10.18637/jss.v074.i11>
- Qian, Z., Cebere, B.-C., & van der Schaar, M. (2023). Synthcity: Facilitating innovative use cases of synthetic data in different data modalities. <https://doi.org/10.48550/arXiv.2301.07573>
- SynFinder: Source code and method catalog [Evaluated revision 89e0923fb1d5dcac5e676d28f207602f011ee47d]. (2026). <https://github.com/sunchang0124/SynFinder>
- Walonoski, J., Kramer, M., Nichols, J., Quina, A., Moesel, C., Hall, D., Duffett, C., Dube, K., Gallagher, T., & McLachlan, S. (2018). Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record. *Journal of the American Medical Informatics Association*, 25(3), 230–238. <https://doi.org/10.1093/jamia/ocx079>
- Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling tabular data using conditional GAN. <https://doi.org/10.48550/arXiv.1907.00503>